Building a Boilerplate vs. Substantive Sentence Classifier

Course assignment — NLP on earnings-call transcripts

Objective

Design, train, and evaluate a classifier that labels each sentence of an earnings-call transcript as either **boilerplate** (scripted intros, safe-harbor language, operator/analyst housekeeping, generic thanks) or **substantive** (material numbers, guidance, segment commentary, strategy, Q&A; specifics). Benchmark several classifier families, pick a winner under a recall constraint on the substantive class, and ship a small GUI that lets a reader see the tagging inline on a transcript.

Starting materials

- A directory of earnings-call transcripts (.txt). You already have these; they become your raw corpus.
- Access to Python, scikit-learn, and at least one local or hosted LLM for gold labeling. A local Ollama installation with qwen3.6 (or similar) and mxbai-embed-large is recommended. Anthropic or OpenAI APIs are also acceptable.
- Freedom to choose any additional libraries you want (SetFit, FastText, floret, FinBERT, SentenceTransformers, etc.).

What you must turn in

Submit **one zip archive** containing the four artifacts below. Incomplete submissions will be graded only on what is present.

1. **Notebook / scripts** that reproduce your entire pipeline end-to-end: sentence extraction, gold-label creation, training of every classifier you benchmarked, threshold tuning, evaluation, and the final saved model.
2. **Trained best model** (a pickle file, SetFit directory, or equivalent) that your GUI loads at startup.
3. **GUI application** (Streamlit, Gradio, React+Flask, or similar) that can load a transcript and display the tagging inline.
4. **A project write-up PDF** (this handout lists what it must contain). Treat it as a short engineering report, not a transcript of what you did hour by hour. 5–10 pages with figures is appropriate.

Focus items — pay special attention

Four items carry the most weight in the rubric. Read them carefully before you start — they dictate the order in which you should tackle the work.

1. The gold standard must be *correct*

Every quality claim you make about any classifier is only as credible as the labels you measured it against. A noisy gold set hides real gains and invents phantom ones. Your write-up must **explain how you achieved high label quality** — not just "I used an LLM."

- Use more than one labeling source and require agreement. A trio-of-judges majority vote (e.g. three distinct LLMs, or an LLM plus two human passes) is a good pattern. Report how often the judges disagreed.
- Write a clear labeling rubric with anchor examples of both classes, including edge cases you expect to trip up the judges (analyst name intros, generic thanks, one-word answers, mixed sentences).
- Hand-audit a stratified sample of the disagreements. Correct the obvious ones; for genuinely ambiguous cases, document the rule you adopted.
- Freeze a held-out test split and **do not touch it** until your final run.

2. Substantive recall first, boilerplate F1 second

Downstream systems care much more about *losing* substantive information than about occasionally forwarding a boilerplate line. Your classifier must hit **substantive recall** ≥ **0.96** on the validation set; only thresholds that satisfy this floor are eligible. Among the feasible thresholds, maximize **macro-F1** (which rewards boilerplate F1).

- Tune the decision threshold on *validation* (or on out-of-fold probabilities from k-fold cross-validation). Report the threshold you picked and its variance across folds.
- If no threshold meets the recall floor, your classifier fails this criterion. Pick a better classifier or collect more gold labels — don't silently relax the constraint.
- Report precision / recall / F1 per class on the held-out test set. Include the confusion matrix.

3. A leaderboard of accuracy and speed

You must benchmark at least **five** materially different classifier families and present them side-by-side. For each entry, report test accuracy, macro-F1, per-class F1, training time, and approximate inference throughput (sentences per second on the same machine). Order by macro-F1 descending.

Family	Why try it	Typical cost
Rules + regex	Trivially fast, catches textbook safe-harbor / operator phrases.	Seconds
Linear model on frozen embeddings	Strong sentence-embedding + LogReg baseline; easy to explain.	Seconds
Tree ensembles on enriched features	RandomForest / HistGBM on embeddings + hand-crafted regex flags.	Minutes
FastText / floret	N-gram bag-of-words, no embedding dependency; tiny model size.	Seconds
Fine-tuned transformer (FinBERT)	Domain-adapted BERT fine-tuned on your gold labels.	10–15 min

Family	Why try it	Typical cost
Contrastive fine-tuning (SetFit)	Usually the strongest single model for small labeled sets.	5–15 min
Ensemble	Mean or rank-averaged probabilities across your top-K members.	Negligible

You are free to add others (SVM-RBF, kNN on cosine, MLP, LSTM, prompt-based LLM classification, etc.). Ensembles count as one entry each; a rank-averaged ensemble and a mean-probability ensemble are **both** fine to include.

4. A GUI for inline tagging

Ship a runnable user interface that lets someone load an earnings-call transcript and see the classifier's tags inline, in the original document flow. Minimum features:

- File picker or textarea for pasting a transcript.
- A single **document view** showing every sentence in its original position. Boilerplate sentences rendered with a distinct background (red is the convention in this course); substantive sentences rendered plainly.
- A **statistics panel** showing boilerplate count, substantive count, and percentages.
- A run command in your write-up so the grader can start the UI in under a minute.

Streamlit or Gradio are the shortest paths. A React + FastAPI stack is also fine if you're comfortable with it. Polish matters less than correctness.

Suggested workflow

You do not have to follow this order, but every step listed here needs to happen somewhere in your pipeline.

1. **Extract sentences.** Read each transcript, de-duplicate repeated lines, split on paragraph breaks, then sentence-tokenize (NLTK punkt is standard). Drop sentences shorter than ~40 characters.
2. **Build the gold set.** Sample 2,000–3,000 sentences, label each one with your judging pipeline (multi-LLM majority vote is the recommended approach), audit disagreements, freeze.
3. **Stratified split.** 60% train / 20% val / 20% test, stratified by label. Set a random seed so your results are reproducible.
4. **Hand-crafted features.** Beyond the embedding, add 20–30 regex flags that are meaningful for earnings-call transcripts: speaker-intro patterns, analyst firms, operator cues, safe-harbor phrases, digit / dollar / percent counts, generic-thanks detectors, sentence length, punctuation ratios.
5. **Classifier zoo.** Train every family from the table on the same feature matrix. Use 5-fold out-of-fold probabilities on the train+val pool to pick a threshold that meets the recall constraint; evaluate once on test.
6. **Ensembles.** Mean-probability and rank-averaged ensembles of your top-5 non-transformer members are usually worth an extra 1–3 F1 points.
7. **Pick the winner.** Under the recall constraint, highest macro-F1 on the held-out test set wins. Save the model and its threshold to disk.
8. **GUI.** Wire the saved model into your GUI. Verify on 2–3 unseen transcripts that the tagging looks sane.
9. **Write up.** What you did, why it worked, what surprised you. Include the leaderboard, a screenshot of the GUI, and a note on what you would try next with more time.

What the write-up PDF must contain

- **Introduction** — one paragraph on the problem and your framing.
- **Gold-standard methodology** — labeling sources, rubric, disagreement rate, audit process, final class balance.
- **Feature engineering** — which regex flags you built and why.
- **Classifier-zoo results** — the leaderboard table, plus a short paragraph per family describing its strengths and failure modes on this data.
- **Recall-constrained threshold selection** — the threshold-tuning procedure, per-class precision/recall/F1, and confusion matrices for your best model.
- **Error analysis** — a handful of misclassifications by direction, with commentary on whether they are label noise, feature gaps, or genuinely hard cases.
- **GUI screenshot** — one full-page screenshot showing a transcript tagged inline with the statistics panel visible.
- **Reproducibility** — exact commands to recreate your best model and start the GUI from a clean checkout.

Grading rubric (100 points)

Criterion	Points	What we look for
Gold-standard quality	25	Multi-source labels, rubric with anchors, audited disagreements, documented ambiguity rules.
Substantive recall ≥ 0.96 on test	15	Hard floor. Failing it caps this line at 0 and no partial credit.
Macro-F1 on test (boilerplate & substantive)	20	Higher F1 scores better. Relative to the class leaderboard; ~0.90 is the target.
Leaderboard breadth and fairness	15	At least five materially different families, same features and splits, speed numbers reported.
GUI	10	Loads a transcript, renders sentences inline, highlights boilerplate, shows statistics. Runs in under a minute.
Write-up clarity and honesty	15	Clean prose. Honest error analysis. You acknowledge what didn't work.

Ground rules

Collaborating on ideas and debugging is encouraged. Sharing code, gold labels, or write-up text is not. Cite every library, pretrained model, and online resource you used. If you used an LLM to help write any section of the report, say which section and how — this is not cheating, but undocumented LLM prose is.

Tips and common pitfalls

- **Cache your embeddings.** Embedding 2,000 sentences through a local model is slow; pickle the (train, val, test) embedding matrices once and reuse them across every classifier.
- **Cache your gold labels.** Re-running an LLM judge over 2,500 sentences costs time and, if hosted, money. Write your judge outputs to Parquet and resume on interruption.
- **Separate the sentence pool from the gold pool.** Your extraction step may yield 10,000+ sentences; you'll only label a sample. Keep both on disk.
- **Tune thresholds on OOF, not on a single val split.** Single-split thresholds jitter ±0.1 between re-runs; OOF-pooled thresholds are much more stable. Report the fold-to-fold standard deviation.
- **Don't confuse sentence probability with the threshold.** A model with mean P(boilerplate) = 0.25 does not need a 0.5 threshold. Tune it.
- **Test your GUI on transcripts the model has never seen.** Verifying on training transcripts is a tempting but misleading demo.

Suggested libraries

Library	Purpose
scikit-learn	Core classifiers, splits, metrics, thresholding.

Library	Purpose
nltk (punkt)	Sentence tokenization.
sentence-transformers	Frozen-embedding baselines.
setfit	Contrastive fine-tuning on small labeled sets.
transformers + datasets	FinBERT / general transformer fine-tuning.
fasttext-wheel, floret	Fast n-gram text classifiers, tiny models.
pandas, numpy, pyarrow	Data wrangling and parquet caching.
plotly or matplotlib	Leaderboard / confusion-matrix figures.
streamlit or gradio	Quickest GUI path.
ollama / anthropic / openai	Your gold-labeling judges.

Submission

Turn your zip in to Howard in the same way you submitted your last assignment.